

Mantenimiento del Software.

1. Introducción

Frente a la considerable velocidad con que se ha desarrollado el Hardware, el desarrollo del Software ha sufrido un retraso histórico en cuanto a la elaboración y disposición de un cuerpo de doctrina tecnológico (metodologías y herramientas) y científico (modelos y teorías en los que basar lo anterior). A finales de los setenta ya se había popularizado el término Crisis del Software para referirse a esta situación. Los síntomas de esta crisis han estado repercutiendo desde entonces en la industria de desarrollo de software y aún se sienten sus efectos. Para resolver el problema ha surgido un área de la informática llamada Ingeniería del Software.

Una de las causas principales de esta situación ha sido la poca importancia dada al proceso de Mantenimiento de Software desde todos los colectivos: gestores de empresas, responsables de Centros de Proceso de Datos, informáticos y usuarios.

Definición de Mantenimiento

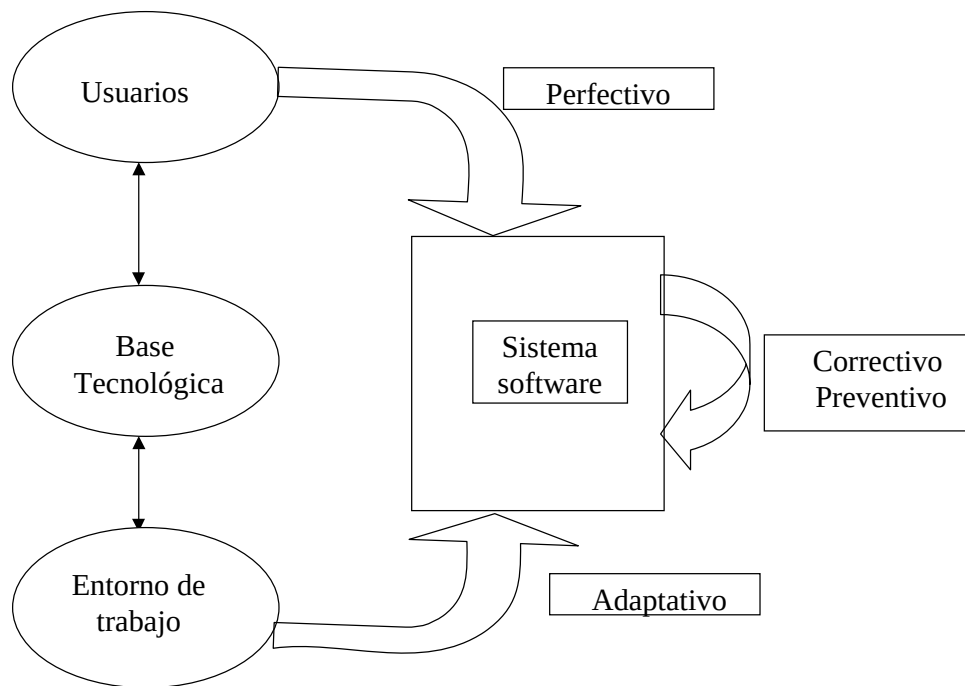
El estándar IEEE 1219 define el Mantenimiento del Software como “la modificación de un producto de software después de haber sido entregado al cliente o usuario, con el fin de corregir defectos, mejorar el rendimiento u otras propiedades deseables, o para adaptarlo a un cambio en el entorno”.

En el estándar ISO 12207, de Procesos de Ciclo de Vida del Software, se establece que el “Proceso de Mantenimiento contiene las actividades y tareas realizadas por el mantenedor. Este proceso se activa cuando el producto software sufre modificaciones en el código y la documentación asociada, debido a un problema o a la necesidad de mejora o adaptación. El objetivo es modificar el producto software existente preservando su integridad”.

Otra definición similar a las anteriores dice que “la fase mantenimiento se centra en el cambio que va asociado a la corrección de errores, a las adaptaciones requeridas a medida que evoluciona el entorno del software, y a los cambios debidos a las mejoras producidas por los requisitos cambiantes del cliente”.

2. Tipos de mantenimiento

En las anteriores definiciones de mantenimiento aparecen indicados, directa o indirectamente, cuatro tipos de mantenimiento: correctivo, adaptativo, perfectivo y preventivo, que suelen ser los referenciados en la bibliografía. La siguiente figura refleja un resumen del papel que representa cada tipo de mantenimiento. Puede observarse que, mientras que el cambio tecnológico afecta indirectamente a los sistemas software, el entorno de trabajo y los usuarios lo hacen directamente, produciendo demandas de mantenimiento adaptativo y perfectivo respectivamente.



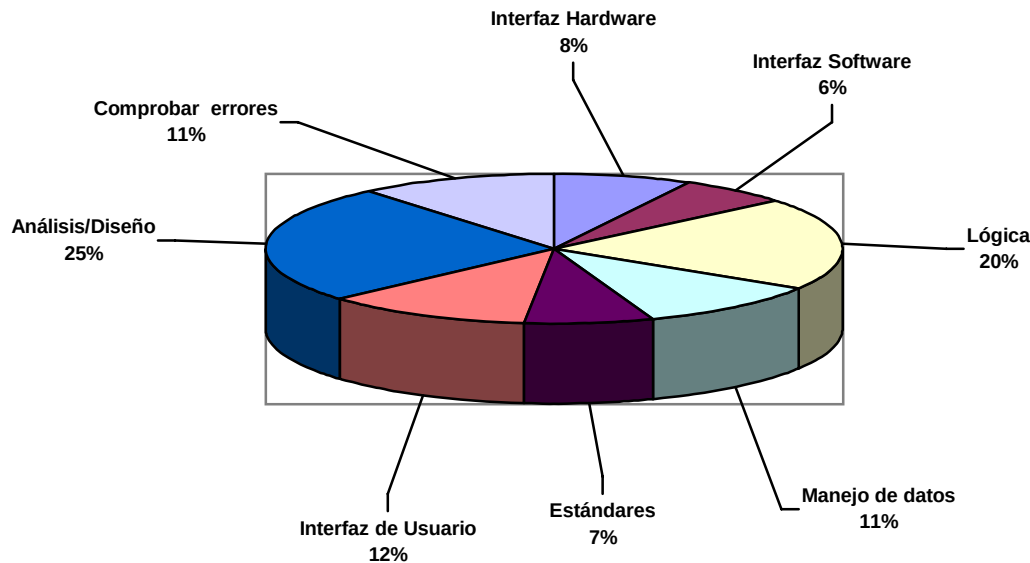
Las fuentes del mantenimiento del Software

Mantenimiento Correctivo

El mantenimiento correctivo tiene por objeto localizar y eliminar los posibles defectos o errores de los programas. Un defecto en un sistema es una característica del sistema con el potencial de causar un fallo. Un fallo ocurre cuando el comportamiento de un sistema es diferente al establecido en las especificaciones. Entre otros, los fallos en el software pueden ser de:

- Procesamiento, por ejemplo, salidas incorrectas de un programa.
- Rendimiento, p. e., tiempo de respuesta demasiado alto en una búsqueda de información.
- Programación, por ejemplo, inconsistencias en el diseño de un programa.
- Documentación, p. e., inconsistencias entre la funcionalidad de un programa y el manual de usuario.

En la siguiente figura se muestra la distribución de las causas de los defectos del Software. Según dicho estudio estadístico el 37,4% de los defectos se origina en la fase de especificación de requisitos, el 25,5% en la fase de diseño y el 36,6% en la fase de codificación.



Origen de los defectos del software.

Mantenimiento Adaptativo

El mantenimiento adaptativo consiste en la modificación de un programa debido a cambios en el entorno (hardware o software) en el cual se ejecuta. Estos cambios pueden afectar al S.O. (cambio a uno más moderno), a la arquitectura física del sistema informático (paso de una arquitectura de red de área local a Intranet/Internet) o al entorno de desarrollo del software (incorporación de nuevos elementos o herramientas como ODBC). El cambio puede ser desde muy pequeño (retocar la estructura de un módulo) hasta muy grande (reescribir prácticamente todo el programa para su ejecución en un entorno de red).

Los cambios en el entorno software pueden ser de dos clases:

- En el entorno de los datos, por ejemplo, al dejar de trabajar con un sistema de ficheros y sustituirlo por un SGBD relacional.
- En el entorno de los procesos, por ejemplo, migrando a una nueva plataforma de desarrollo con componentes distribuidos, Java; ActiveX, etc.

Este tipo de mantenimiento es cada vez más usual debido al cambio, cada vez más rápido, en la informática (nuevas generaciones de Hardware cada dos años, nuevos S.O. que se renuevan regularmente, mejoras en los periféricos o en otros elementos del sistema, etc). Frente a esto, la vida útil de un sistema software puede superar fácilmente los diez años.

Mantenimiento Perfectivo

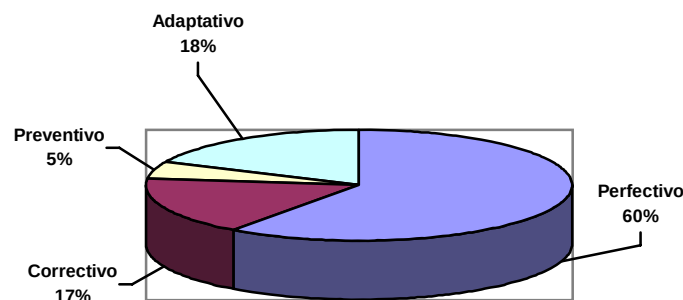
Este tipo de mantenimiento viene motivado por cambios en la especificación, normalmente debidos a cambios en los requisitos de un producto software. La casuística de estos cambios es muy variada,: desde algo tan simple como cambiar un formato de impresión de un informe, hasta la incorporación de un nuevo módulo aplicativo. Podemos definir el mantenimiento perfectivo como el conjunto de actividades para mejorar o añadir nuevas funcionalidades requeridas por el usuario.

Este tipo de mantenimiento aumenta cuando un producto de software tiene éxito comercial y es utilizado por muchos usuarios, ya que cuanto más se utiliza un software, más peticiones de los usuarios se reciben demandando nuevas funcionalidades o mejorando las existentes.

Mantenimiento Preventivo

El mantenimiento Preventivo consiste en la modificación del software para mejorar las propiedades del software sin alterar sus especificaciones funcionales. Por ejemplo, se pueden incluir sentencias que comprueben la validez de los datos de entrada, reestructurar los programas para mejorar su legibilidad, o bien incluir nuevos comentarios que faciliten la posterior comprensión del programa. Este tipo de mantenimiento es el que mejor partido saca de las técnicas de ingeniería inversa y reingeniería.

En algunos casos se ha planteado otro tipo de mantenimiento para la Reutilización, consistente en modificar el software (buscando y modificando componentes para incluirlos en bibliotecas) para que sea más fácilmente reutilizable; pero en realidad este tipo de mantenimiento es Preventivo, especializado en mejorar la reusabilidad del software.



Costes relativos de cada tipo de mantenimiento

3. Actividades del mantenimiento

El desconocimiento de las actividades que implica el mantenimiento del software puede inducir a infravalorar su importancia, y se tiende a asociar con la corrección de errores en los programas. Por esta causa, la impresión más generalizada es que la mayor parte del mantenimiento es el correctivo, pero como hemos visto en el gráfico anterior los mayores porcentajes de esfuerzo se dedican a mantenimiento perfectivo.

Algunos autores han identificado diferentes tipos de actividades que se realizan en cada modificación del software, en concreto podemos ver las siguientes:

- Análisis del impacto y de costes/beneficios: esta actividad se dedica a analizar diferentes alternativas de implementación y/o a comprobar su impacto en la planificación, coste y facilidad de operación.
- Comprensión del cambio: puede consistir en localizar el error y determinar su causa, o bien en comprender los requisitos de una mejora solicitada.
- Diseño del cambio: se refiere al diseño propuesto para el cambio, pudiéndose incluir un rediseño del sistema.
- Codificación y pruebas unitarias: se codifica y prueba el funcionamiento de cada componente modificado.
- Inspección, certificación y consultoría: esta actividad se dedica a inspeccionar el cambio, comprobar otros diseños, reuniones de inspección, etc.
- Pruebas de integración: se refiere a comprobar la integración de los componentes modificados con el resto del sistema.
- Pruebas de aceptación: en esta actividad, el usuario comprueba junto al personal encargado del mantenimiento, la adecuación del cambio a sus necesidades.
- Pruebas de regresión: en esta actividad se somete el Software modificado a casos de pruebas previamente almacenados y por los que ya pasó.
- Documentación del sistema: se revisa y reescribe, en caso necesario, la documentación del sistema para que se ajuste al producto software ya modificado.
- Otra documentación (del usuario por ejemplo): se revisan y reescriben, en caso necesario, los manuales de usuario y otra documentación, excepto la del sistema.
- Otras actividades: como las dedicadas a la gestión del proyecto de mantenimiento.

Con el fin de mostrar más claramente la distribución del esfuerzo durante las modificaciones, se clasifican las actividades anteriores en cinco grupos:

Grupo	Actividades	% Esfuerzo
Análisis y comprensión	Análisis de impacto y de costes/beneficio Comprensión del cambio	13%
Diseño	Diseño del cambio 50% de inspección, certificación y consultoría	16%
Implementación	Codificación y pruebas unitarias 50% de inspección, certificación y consultoría	29%
Pruebas	Pruebas de integración Pruebas de aceptación Pruebas de regresión	24%
Otras	Documentación del sistema Otra documentación Otras actividades	18%

4. Costes del mantenimiento

Está comprobado estadísticamente que el coste de mantenimiento de un sistema de software a lo largo de su vida útil supone más del doble de los costes de desarrollo. La tendencia es creciente con el paso del tiempo, tal como se ve en la siguiente tabla, en la cual se indica el porcentaje que supone el mantenimiento respecto al coste total.

Fechas	% Mantenimiento
Años 70	35%-40%
1976	60%
Años 80	60%
1987	67%
1985-1989	75%
1990	80%
Años 90 (prev.)	90%

Los programadores pasan el 61% de su vida profesional realizando trabajos de mantenimiento, y sólo un 39% en nuevos desarrollos. Hay algunas empresas en las que estos porcentajes se acercan al 95% de los recursos destinados al mantenimiento, con lo cual se hace imposible el desarrollo de nuevos productos software.

Los estudios sobre la situación del mercado comercial del mantenimiento del software son escasos:

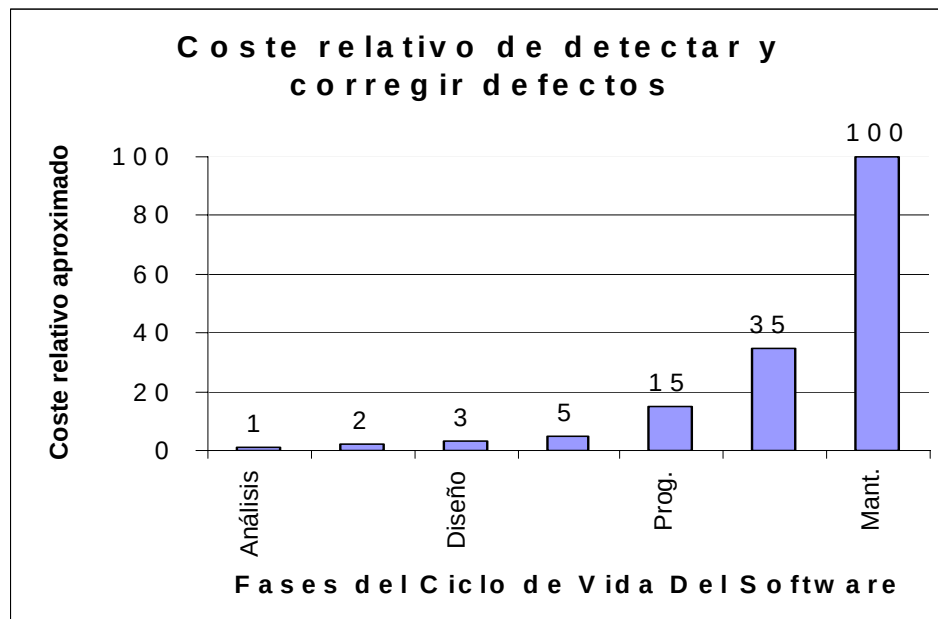
En Francia (1.991) 48.000 millones de francos en gastos de software, de los cuales:
34.000 millones de francos en mantenimiento (más del 70%).

En España (Estudio sobre TI del MINER, 1.998)
41.793 millones de pesetas en mantenimiento .
53.708 millones de pesetas en externalización.

Causas que influyen en los trabajos de mantenimiento.

- Software desarrollado hace más de 10 ó 20 años (código heredado). Aunque estos programas fuesen creados con las mejores técnicas de diseño y codificación de su momento (y la mayoría no lo fueron), se construyeron con restricciones de tamaño y espacio de almacenamiento, y se desarrollaron con herramientas tecnológicamente limitadas.
- Estos programas han sufrido una o varias migraciones a nuevas plataformas o S.O.
- Han experimentado múltiples modificaciones para mejorarlos y adaptarlos a las nuevas necesidades de los usuarios. Todos estos cambios se realizaron sin tener en cuenta la arquitectura general del sistema, y el resultado es de unos sistemas software que tienen que seguir funcionando en la actualidad con una baja calidad.

Una causa directa de los grandes costes del mantenimiento es que el coste relativo de reparar un defecto aumenta considerablemente en las últimas etapas del ciclo de vida del software, de forma que la relación entre el coste de detectar y reparar un defecto en la fase de análisis de requisitos y en la fase de mantenimiento es de 1 a 100 respectivamente, según la siguiente figura:



Razones por las que es menos costoso detectar y corregir un error en las etapas iniciales del ciclo de vida del software:

- Es más fácil cambiar la documentación que modificar el código.
- Un cambio en una fase tardía puede requerir que sea modificada la documentación de todas las fases anteriores.
- Es más fácil detectar un defecto durante la fase en que se ha introducido que en etapas posteriores
- El coste aproximado de reparar un defecto aumenta considerablemente en las últimas etapas del ciclo de vida del software.

Se esconden gran cantidad de problemas potenciales y costes encubiertos dentro de los costes de mantenimiento. Un coste intangible del mantenimiento del software se encuentra en las oportunidades de desarrollo que se han de posponer o que se pierden, debido a que los recursos disponibles están dedicados a las tareas de mantenimiento. Otros costes intangibles pueden ser:

- Insatisfacción del cliente cuando no se puede atender en un tiempo razonable una petición de reparación o modificación.
- Errores ocultos introducidos al cambiar el software durante el mantenimiento que disminuyen la calidad del producto.
- Perjuicio en otros proyectos al tener que dejarlos total o parcialmente para atender peticiones de mantenimiento.

En suma, el coste final de mantenimiento de software es igual a los recursos empleados en tareas de mantenimiento + reducción de productividad en otros sistemas en desarrollo.

5. Dificultades del mantenimiento

El problema de mantenimiento se resume en realizar el mantenimiento del software de forma tan rigurosa que la calidad no se deteriore como resultado de este proceso. A continuación veremos como debe mantenerse el software para preservar su fiabilidad.

o **Código heredado** (Legacy Code).

La mayor parte del código actual pertenece a aplicaciones desarrolladas hace algún tiempo, con técnicas y herramientas en desuso y por personas que probablemente ya no se encargan de su mantenimiento. En muchas ocasiones el código heredado ya ha sido objeto de múltiples actividades de mantenimiento, y reescribirlo resulta en ocasiones inadecuado por la gran carga financiera que supuso su desarrollo y la necesidad de amortizarlo.

Los problemas específicos del mantenimiento de código heredado han sido caracterizados en las llamadas *Leyes del Mantenimiento del Software*:

- Continuidad del cambio. Un programa que se utilice en el mundo real ha de cambiar para que pueda seguirse usando.
- Incremento de la complejidad. A medida que se realizan cambios, la estructura se hace más compleja.
- Evolución del programa. La evolución de un programa es un proceso autoregulado. Las medidas de ciertas propiedades (tamaño, tiempo entre versiones, número de errores) revelan la existencia de tendencias e invariantes.
- Conservación de la Estabilidad Organizacional. La carga que supone el desarrollo de un programa es aproximadamente constante e independiente de los recursos dedicados.
- Conservación de la familiaridad. El incremento de cambios incluidos en cada versión es (release) aproximadamente constante.
- Falta de metodologías.
- Pérdida de estructuración.
- Aumento de la dificultad de efectuar cambios.
- Poca participación de los usuarios.
- Consideración del trabajo de mantenimiento como una actividad inferior.
- Efectos secundarios. Una de las principales dificultades del mantenimiento de software es el riesgo del llamado efecto bola de nieve, de manera que los cambios producidos por una acción de mantenimiento introducen efectos secundarios que provocarán nuevas peticiones de mantenimiento en el futuro.

6. Efectos secundarios del mantenimiento

6.

Al cambiar un procedimiento lógico tan complejo como el que constituye la mayor parte de los programas actuales, la probabilidad de error es muy grande. Por esta razón, una de las principales dificultades del mantenimiento del software es el riesgo del efecto Bola de Nieve, de manera que los cambios producidos por una petición de mantenimiento introducen efectos secundarios que implicarán nuevas peticiones de mantenimiento en el futuro. Estos efectos secundarios suponen nuevos defectos que aparecen como consecuencia de las modificaciones realizadas.

Según las consecuencias que se derivan, los efectos secundarios del mantenimiento del software son de tres clases:

1. Efectos secundarios sobre el código

Las consecuencias de un simple error de mecanografía de un signo de puntuación en otro, o el olvido de un “;” pueden ser muy importantes y sirven para corroborar que los efectos secundarios por los cambios en el código son difíciles de prever. Las modificaciones en el código fuente con más probabilidad de inducir a nuevos errores son:

- Cambios en el diseño que suponen muchos cambios de código.
- Eliminar o modificar subprogramas.
- Eliminar o modificar etiquetas.
- Eliminar o modificar identificadores.
- Cambios para mejorar el rendimiento.
- Modificar la apertura/cierre de ficheros.
- Modificar operaciones lógicas.

2. Efectos secundarios sobre los datos

Las estructuras de datos son una parte fundamental y básica del software, por lo que cualquier cambio en ellas puede conducir a fallos importantes en el sistema. Los efectos secundarios de este tipo pueden aparecer debido a los siguientes cambios:

- Redefinición de constantes locales o globales.
- Modificar los formatos de registros o archivos.
- Cambio en el tamaño de una matriz o estructura similar.
- Modificación de la definición de variables globales.
- Reinicialización de indicadores de control o punteros.
- Cambios en los argumentos de los subprogramas.

Para reducir estos efectos secundarios es importante documentar todos los datos, incluyendo tablas de referencias cruzadas que los asocien con los subprogramas que los usan.

3. **Efectos secundarios sobre la documentación**

Se producen cuando los cambios sobre el código de la aplicación no se reflejan en la documentación de diseño o de usuario. Si la documentación técnica no se corresponde con el estado actual del software, se producirán efectos secundarios debidos a una incorrecta caracterización de las propiedades del software. Los cambios que con mayor probabilidad pueden producir efectos secundarios sobre la documentación son:

- Modificar el formato de las entradas interactivas.
- Nuevos mensajes de error no documentados.
- Tablas o índices no actualizados.
- Texto no actualizado correctamente.

Es muy recomendable revisar la configuración entera del software, incluyendo la documentación para evitar estos efectos secundarios.

7. Soluciones al problema del Mantenimiento

Las diversas propuestas para resolver este problema pueden dividirse en dos categorías: las que proponen soluciones de gestión (organizativas) y las que proponen soluciones técnicas (metodologías y herramientas).

o **Soluciones de gestión.**

El mantenimiento del software se considera desde el punto de vista financiero como:

- Un continuo consumidor de recursos.
- Un proceso con beneficios potenciales oscuros y poco cuantificados.

Esta situación puede ser evitada si los gestores de las organizaciones son conscientes de:

- La importancia de las tecnologías de la información para las organizaciones.
- El software es un activo corporativo que puede suponer una ventaja competitiva.

Los gestores deberán adquirir un compromiso visible con algunas soluciones organizativas.

- Recursos dedicados al mantenimiento. El recurso clave para el mantenimiento del software es el humano, por tanto una manera de mejorar el mantenimiento podría ser constituir un grupo separado de programadores dedicado a mantener código antiguo.
- Gestión de la calidad. Para resolver el problema a largo plazo es necesario adoptar técnicas que permitan mejorar la calidad del proceso en su conjunto. Entre las mejores técnicas de gestión de la calidad del software se incluyen:

1. Uso de técnicas estándares para la descomposición del software en entidades funcionales.

2. Empleo estricto de estándares de documentación del software.
 3. Diseño paso a paso en cada nivel de descomposición del software.
 4. Uso de código estructurado.
 5. Definición de todas las interfaces y estructuras de datos importantes antes de comenzar el diseño detallado.
 6. Uso de métricas de productos y procesos, así como herramientas de desarrollo de software (por ejemplo: entorno único que integre editor, compilador y depurador).
- Gestión estructurada del mantenimiento. Este mantenimiento estructurado aparece como resultado de la aplicación de una metodología de Ingeniería del Software. La existencia de una adecuada Configuración del Software (documentación e información sobre los requerimientos, especificación, diseño y pruebas) reduce el esfuerzo requerido en el mantenimiento y mejora la calidad general de los cambios.

Si el mantenimiento no es estructurado se sufren las consecuencias de la falta de metodología:

- Evaluación del código poco legible en algunos casos.
- Complicada comprensión del sistema por la pobre documentación interna.
- Dificultad para descubrir las consecuencias de los cambios en el código.
- Imposibilidad de realizar pruebas de regresión (anteriores) por no existir un registro de pruebas.

Cuando no hay más remedio que mantener Código Heredado, las dificultades pueden atenuarse siguiendo algunas sugerencias propuestas por Yourdon [1975]:

- Prevenir antes de curar (es decir, obtener la mayor información posible sobre el programa antes de que surjan las emergencias de mantenimiento).
- Conocer y extender el flujo de control general del programa, dibujando los diagramas de estructura y de flujo de alto nivel.
- Evaluar la documentación.
- Añadir comentarios al código para facilitar su posterior entendimiento.
- Usar las ayudas que proporcionan los compiladores: listado de referencias cruzadas, tablas de símbolos, etc.
- Respetar en lo posible el estilo y forma previos al realizar cambios.
- Señalar en el código las instrucciones cambiadas.
- Asegurarse antes de eliminar el código (guardando copias).
- Usar variables propias que eviten efectos secundarios al utilizar variables ya existentes.
- Llevar un registro completo de todas las actividades de mantenimiento.
- Añadir comprobación de errores.

- Organización del equipo humano. Es importante que se establezca una organización del equipo de mantenimiento, estableciendo claramente las personas que participarán en cada actividad para evitar que el mantenimiento se realice “como se pueda”. En esta organización se deberán establecer claramente los procedimientos de evaluación, control, supervisión e información de cada petición de mantenimiento.

Una posibilidad de organización del equipo de mantenimiento es distinguir tres organizaciones lógicas involucradas en el proceso de mantenimiento, con los siguientes perfiles:

- a) *Cliente:* es la organización propietaria del software y que recibe el servicio de mantenimiento. Tiene los siguientes perfiles:
 - *Solicitante:* quien presenta las solicitudes de modificación. Establece los requerimientos necesarios para su implementación y los entrega a la organización de mantenimiento.
 - *Organización del sistema:* es el departamento que conoce el sistema que será mantenido.
 - *Atención a los usuarios:* es el departamento que presta la asistencia a los usuarios.
- b) *Organización de mantenimiento:* es la organización que realiza el servicio de mantenimiento. En esta organización se identifican cuatro perfiles:
 - *Gestor de peticiones:* acepta o rechaza las peticiones de modificación y decide el tipo de mantenimiento que debe aplicarse.
 - *Planificador:* planifica la cola de peticiones de modificación aceptadas.
 - *Equipo de mantenimiento:* es el grupo de personas que implementa la solicitud de modificación.
 - *Responsable de mantenimiento:* prepara la etapa de mantenimiento y establece las normas y procedimientos necesarios para llevar a cabo la metodología de mantenimiento usada.
- c) *Usuario:* es la organización que utiliza el software objeto de mantenimiento. En esta organización sólo está el perfil Usuario.

- Documentación de los cambios. Es muy importante realizar una correcta documentación de los cambios. Por ello es conveniente que las peticiones de mantenimiento se hagan usando un formulario estandarizado, y que el equipo de mantenimiento elabore un Informe de cambios para cada petición de mantenimiento que incluirá un estudio del esfuerzo requerido para satisfacer la petición, la naturaleza de las modificaciones necesaria, y la prioridad del cambio.

La mayoría de autores coinciden al detallar el conjunto de informaciones que deben recogerse en cada cambio, que suelen ser las siguientes:

1. Descripción del cambio.
 - Localización
 - Subsistemas afectados.
 - Módulos afectados.
 - Entradas/Salidas afectadas.

- Tamaño
 - Líneas de código añadidas, modificadas y eliminadas.
 - Módulos examinados, añadidos, modificados y eliminados.
 - Tipo de cambio.
 - Correctivo.
 - Perfectivo.
 - Preventivo.
 - Adaptativo
- 2. Descripción del proceso de cambio.
 - Esfuerzo dedicado.
 - Experiencia del personal de mantenimiento.
 - Tiempo que ha pasado el personal de mantenimiento trabajando en el sistema.
 - Tiempo que ha pasado el personal de mantenimiento trabajando en el dominio de la aplicación.
 - Si el cambio generó documentación, relacionarla.
- 3. Descripción del problema.
 - Descripción del error.
 - Causa y origen del error.
 - Momento del proceso en que se produjo el error.
 - Dificultad.
 - Causas que dificultaron la modificación.
 - Actividad más difícil relacionada con la modificación.
 - Cantidad de esfuerzo desperdiciado.
 - Decisiones que se podrían haber tomado para disminuir la dificultad de los errores.

o **Soluciones técnicas.**

Son fundamentalmente de dos tipos: Herramientas y Métodos

1. Herramientas: diseñadas para ayudar al personal de mantenimiento a comprender el programa y probar sus modificaciones para asegurar que no se han introducido errores.
2. Métodos: los principales métodos usados en el mantenimiento del software son:
 - **Reingeniería**: [Bennett, 1990] consiste en el examen y modificación de un sistema para reconstruirlo en una nueva forma.
 - **Ingeniería inversa**: [Chikofsky y Cross, 1990] es el proceso de analizar un sistema para identificar sus componentes y las interrelaciones que existen entre ellos, así como para crear representaciones del sistema en otra forma o en un nivel de abstracción más elevado.
 - **Reestructuración del software**: [Arnold, 1986] consiste en la modificación del software para hacerlo más fácil de entender y cambiar y menos susceptible de incluir errores en cambios posteriores. Se diferencia de la ingeniería inversa en que el software reestructurado tiene el mismo nivel de abstracción que el original.

Estas tres técnicas tienen relación entre ellas y a menudo se usan varias conjuntamente.

También se han propuesto otros métodos diferentes de estos tres. Por ejemplo citaremos:

- El método del **Retroajuste Estructurado** [Miller, 1979], definido como “la aplicación de técnicas de programación estructurada de hoy a los sistemas de ayer para atender las demandas del mañana”.
- La **Transformación de Programas** es un método formal para la construcción o modificación de programas propuesto como solución parcial al problema del mantenimiento del software. Parte de una especificación o programa ya existente para obtener otro programa equivalente mediante unas transformaciones sucesivas (cambios realizados en el código del programa de forma que la semántica del programa no cambia). Se caracteriza por preservar la exactitud de cualquier programa al que se aplique este método.

8. Mantenibilidad del software

Veamos algunas definiciones de mantenibilidad del software según algunos organismos:

- Gobierno federal de USA: facilidad con la que el software puede ser mejorado, adaptado o corregido para satisfacer los requerimientos especificados.
- IEEE-1.990: facilidad con que un sistema o componente de software puede ser modificado para corregir defectos, mejorar el rendimiento u otros atributos, o adaptarse a un cambio de entorno.
- ISO 1.994: conjunto de atributos referidos al esfuerzo necesario para realizar las modificaciones especificadas o implicadas.

Los factores que influyen en la mantenibilidad son los siguientes:

- Falta de cuidado en las fases de diseño, codificación o prueba.
- Pobre configuración del producto software.
- Adecuada cualificación del equipo de desarrolladores del software.
- Estructura del software fácil de comprender.
- Facilidad de uso del sistema.
- Empleo de lenguajes de programación y S.O. estandarizados.
- Estructura de la documentación estandarizada.
- Documentación disponible de los casos de prueba.
- Incorporación en el sistema de facilidades de depuración.
- Disponibilidad del hardware adecuado para realizar el mantenimiento.
- Disponibilidad de la persona o grupo que desarrolló originariamente el software.
- Planificación del mantenimiento.

El último factor es el que más influye positivamente en la mantenibilidad del software, ya que si vemos el software como un producto que estará sujeto a cambios, las etapas previas del ciclo de vida pueden realizarse de forma que aumenten las posibilidades de producir software más fácilmente mantenible.

La mantenibilidad del software es una propiedad difícil de cuantificar, pero puede medirse indirectamente evaluando los atributos que la caracterizan. Para ello algunos autores han propuesto diversas clases de métricas de mantenibilidad:

- De esfuerzo: [Gilb, 1979] mediante métricas que indican el tiempo dedicado a las diversas tareas realizadas durante la fase de mantenimiento.
- De complejidad: [Kafura y Reddy, 1987] sugieren métricas de complejidad para evaluar la mantenibilidad.
- De estructura: [Rombach, 1987] analiza la correlación entre la estructura de un programa y su facilidad de mantenimiento.

o Atributos de la mantenibilidad del código fuente.

Estructuras de control		Estructuras de Información		Tipografía, identificación y comentarios	
Sistema	Componente	Sistema	Componente	Sistema	Componente
Modularidad	Complejidad	Tipos de datos globales	Tipos de datos locales	Formato del conjunto de programas	Formato de sentencias
Complejidad	Uso de construcciones estructuradas	Estructuras de datos globales	Estructuras de datos locales	Comentarios en el conjunto de programas	Espaciado vertical
Consistencia	Uso de bifurcaciones incondicionales	Acoplamiento del sistema	Acoplamiento de datos	Separación entre módulos	Espaciado horizontal
Nivel de anidamiento	Nivel de anidamiento	Consistencia del flujo de datos	Integridad de la inicialización	Identificadores	Comentarios intramódulos
Control de acoplamiento	Ambito de las estructuras de control	Consistencia de los tipos de datos	Ambito de los datos	Símbolos	-
Encapsulación	Cohesión	Nivel de anidamiento	-	-	-
Reutilización de módulos	-	Complejidad de la E/S	-	-	-
Consistencia del flujo de control	-	Integridad de la E/S	-	-	-

o Medida de la mantenibilidad.

La facilidad de mantenimiento del software, como la calidad o la fiabilidad, es un término difícil de cuantificar. Podemos evaluar la facilidad de mantenimiento indirectamente, considerando los atributos de la actividad de mantenimiento que se pueden medir.

Tiempo medio para reparación (MTTR): tiempo medio que necesita el equipo de mantenimiento para implementar un cambio y poner de nuevo operativo el sistema modificado.

Para su cálculo es necesario registrar (según el autor GILB):

- Tiempo para identificar el problema.
- Tiempo de retraso administrativo.
- Tiempo para obtener las herramientas de mantenimiento.
- Tiempo para analizar el problema.
- Tiempo para hacer la especificación del cambio necesario.
- Tiempo para realizar el cambio.

Pueden utilizarse otras medidas como:

- Ratio entre el tiempo total para implementar los cambios y el número total de cambios implementados.
- Número de problemas sin resolver.
- Tiempo empleado en problemas no resueltos.
- Porcentajes de cambios que introducen nuevos defectos.
- Número de módulos modificados para implementar el cambio.

o **Estándares.**

ISO 12207

Publicado en 1.995, define el mantenimiento como una parte principal del ciclo de vida.

IEEE 1074

Se detallan el conjunto de actividades que aparecen obligatoriamente en el desarrollo y mantenimiento del software. Distingue entre procesos de gestión de proyectos, predesarrollo, desarrollo, postdesarrollo e integrales.

ISO 9126

Se abordan las características que determinan la calidad del software, tanto del producto como de los procesos para desarrollarlo y mantenerlo.

IEEE 1061

Provee una metodología para establecer requerimientos de calidad e identificar, implementar, analizar y validar métricas de calidad de productos y procesos de software.

IEEE 1219

Detalla un proceso iterativo para gestionar y realizar las actividades de mantenimiento.

o **Principales actividades y tareas según ISO 12207**

Actividades	Tareas
Implementación del proceso	Desarrollar planes de mantenimiento Definir procedimientos de petición de modificación
Análisis de problemas y modificaciones	Implementar la gestión de configuración Evaluar impacto Verificar problema Elaborar alternativas Documentar el problema
Implementación de las modificaciones	Obtener aprobación Determinar objetos a modificar
Revisión y aceptación del mantenimiento	Desarrollar modificaciones Revisar integridad
Migración	Obtener aprobación Asegurar ajuste a la norma Desarrollar plan Notificar la futura migración Ejecutar paralelo Notificar migración Realizar revisión post-operación
Retirada	Archivar datos entorno antiguo Desarrollar plan Notificar futura retirada Ejecutar paralelo Notificar retirada Archivar datos entorno antiguo

Bibliografía:

Mantenimiento del Software, modelos, técnicas y métodos para la gestión del cambio.

Mario Piattini, José Villalba, Francisco Ruiz y otros.

Editorial Ra-Ma (Nov-2000)